

Functions & List Power Tools

Pre-Assessment 1 · 10 questions · 33 points

Name: _____

Date: _____

Score: / 33

This is a **pre-assessment**. It shows what you already know **before** we start this unit, so it is fine if some questions cover things you have not learned yet. Try every question you can. If you have no idea at all, write “**not yet**” and move on.

Work on your own, without a computer. You have about **40 minutes**. When a question asks what a program prints, write exactly what Python would print, one line of output per answer line. If the code would crash, write the name of the error (for example, `TypeError`). Show your thinking; partial credit is given.

1. Two answers at once

2 points

```
def min_max(numbers):  
    ordered = sorted(numbers)  
    return ordered[0], ordered[-1]  
  
low, high = min_max([7, 3, 9, 4])  
print(low, high)  
print(min_max([5, 1]))
```

Write exactly what the program prints.

2. Mini Math Toolkit

3 points

```
def power(base, exp=2):  
    return base ** exp  
  
def describe(n):  
    return f"{n} squared is {power(n)} and cubed is {power(n, exp=3)}"  
  
print(power(3))  
print(power(2, 5))  
print(describe(4))
```

Write exactly what the program prints.

3. Whose variable?

2 points

```
bonus = 5

def add_bonus(score):
    bonus = 10
    return score + bonus

print(add_bonus(20))
print(bonus)
```

Write exactly what the program prints.

4. Test first

4 points

A student wrote these tests **before** writing the function:

TESTS

```
assert to_minutes(2, 15) == 135
assert to_minutes(0, 45) == 45
```

(a) What does Python do when an `assert` line is True? What does it do when it is False? (1 pt)

(b) Write `to_minutes(hours, minutes)` with a docstring so that both tests pass. (2 pts)

(c) Write one more good `assert` test for `to_minutes`. (1 pt)

5. Bug-Proof Calculator

3 points

```
def count_evens(nums):
    """Return how many numbers in nums are even."""
    count = 0
    for n in nums:
        if n % 2 == 0:
            count = count + 1
    return count

assert count_evens([2, 4, 5]) == 2
```

(a) What does `count_evens([2, 4, 5])` actually return?

(1 pt)

(b) What happens when the `assert` line runs?

(1 pt)

(c) Describe the fix.

(1 pt)

6. Number grid

3 points

```
grid = [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]]
print(grid[1])
print(grid[2][0])
print(grid[0][-1] + grid[1][1])
```

Write exactly what the program prints.

7. List power tools

4 points

```
names = ["Ivy", "Max", "Leo"]
scores = [88, 95, 72]
for name, score in zip(names, scores):
    print(name, score)

words = ["kiwi", "fig", "banana"]
print(sorted(words))
print(sorted(words, key=len))
for i, w in enumerate(words):
    print(i, w)
```

Write exactly what the program prints.

8. Unique colors

3 points

```
colors = ["red", "blue", "red", "green", "blue", "red"]
unique = set(colors)
print(len(colors), len(unique))
unique.add("red")
unique.add("pink")
print(len(unique))
print("green" in unique)
```

Write exactly what the program prints.

9. One-line list builders

4 points

(a) What is `[x * 2 for x in [1, 2, 3]]`?

(1 pt)

(b) What is `[n for n in range(10) if n % 3 == 0]`?

(1 pt)

(c) Rewrite this loop as **one** list comprehension:

(2 pts)

```
squares = []
for n in numbers:
    if n % 2 == 0:
        squares.append(n * n)
```

10. Class Leaderboard

5 points

Each player is stored as `[name, score]`:

```
players = [{"Ana", 12}, {"Ben", 30}, {"Cy", 21}]
```

Write a function `leaderboard(players)` that **returns** a list of just the names, from highest score to lowest. For the list above it returns `["Ben", "Cy", "Ana"]`. Then write one `assert` test for it.

END OF ASSESSMENT